Experiment 8

Line Detection via Hough Transformation: Cartesian (m, c) vs. Polar (r, θ) Parameter Spaces

1. Ground Truth Setup, Intensity-Weighted Sampling & Accumulator Grids

```
rng = np.random.default_rng(0); H = W = 500
true_lines = [(0.37, 63), (-1.13, 447), (2.21, -190)]  # (m, c) ground-truth parameters
img = np.zeros((H, W), dtype=np.uint8)
for m, c in true_lines: cv2.line(img, (0, round(c)), (W - 1, round(m * (W - 1) + c)), 255, 1)
# Sample 20% pixels at random; weight votes by normalized intensity w in [0, 1]
n = int(0.20 * H * W); k = rng.choice(H * W, n, replace=False); x, y = k % W, k // W; w = img[y, x] / 255.0
# Parameter grids: Cartesian dm matches dc / W; Polar covers [-diag, diag] x [0, 180 deg]
dc = 5; m_vals = np.arange(-5, 5.001, dc / W); c_min, c_max = -2500, 3000; n_c = (c_max - c_min) // dc + 1
diag = int(np.ceil(np.hypot(H, W))); thetas = np.deg2rad(np.arange(0, 180, 0.5)); r_vals = np.arange(-diag, diag + 1)
```

2. Vectorized Hough Accumulators & Iterative Peeling Algorithm

```
def hough_mc(w):
    acc = np.zeros((len(m_vals), n_c))
    for i, m in enumerate(m_vals):
        idx = np.round((y - m * x - c_min) / dc).astype(int); ok = (idx >= 0) & (idx < n_c)
        acc[i] = np.bincount(idx[ok], weights=w[ok], minlength=n_c)
    return acc

def hough_rt(w):
    acc = np.zeros((len(thetas), len(r_vals)))
    for i, t in enumerate(thetas):
        idx = np.round(x * np.cos(t) + y * np.sin(t)).astype(int) + diag; ok = (idx >= 0) & (idx < len(r_vals))
        acc[i] = np.bincount(idx[ok], weights=w[ok], minlength=len(r_vals))
    return acc

def dist_mc(i, j): return np.abs(y - m_vals[i] * x - (c_min + j * dc)) / np.hypot(1, m_vals[i])
def dist_rt(i, j): return np.abs(x * np.cos(thetas[i]) + y * np.sin(thetas[i]) - r_vals[j])

def peel(hough, dist, min_votes=12, tol=8):
    w_left = w.copy(); acc = hough(w_left); found = []
    while True:
        sm = cv2.boxFilter(acc.astype(np.float32), -1, (3, 3), normalize=False)
        i, j = np.unravel_index(sm.argmax(), sm.shape)
        if sm[i, j] < min_votes: break
        found.append((i, j, sm[i, j])); near = dist(i, j) <= tol
        if w_left[near].sum() == 0: break
        w_left[near] = 0; acc = hough(w_left)  # Invalidate inliers & re-vote without pre-set line count
    return acc, found

acc_mc, peaks_mc = peel(hough_mc, dist_mc); acc_rt, peaks_rt = peel(hough_rt, dist_rt)
est_mc = [(float(m_vals[i]), float(c_min + j * dc)) for i, j, _ in peaks_mc]
est_rt = [(-np.cos(thetas[i])/np.sin(thetas[i]), r_vals[j]/np.sin(thetas[i])) if np.sin(thetas[i]) > 1e-6 else (np.nan, np.nan) for i, j, _ in peaks_rt]
```

3. Experimental Visualizations: Image-Space Detections & Parameter Duals

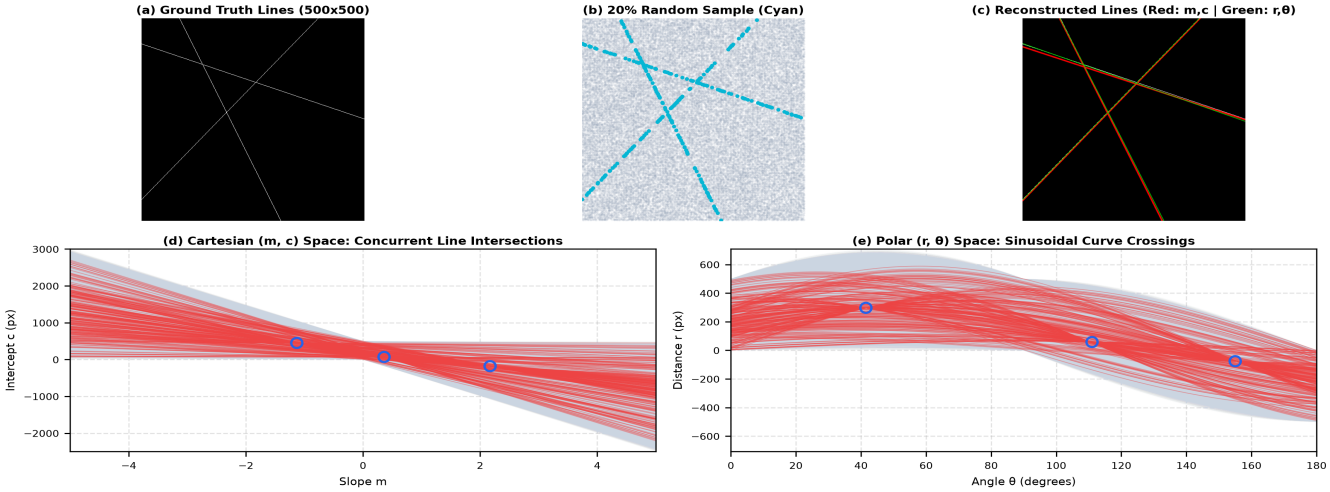


Figure 1: (a) Ground-truth lines, (b) 20% random sample, (c) Reconstructed lines, (d) Cartesian (m,c) line intersections, (e) Polar (r,θ) sinusoidal crossings.

4. Quantitative Benchmark: Ground Truth vs. Recovered Parameters

Target	Ground Truth (m, c)	Cartesian (m, c) Detected	Polar (r, θ) Detected	Polar Abs Error (Δm, Δc)
Line 1	m = 0.37, c = 63.0 px	m = 0.36, c = 70.0 px	m = 0.38, c = 62.1 px	|Δm| = 0.01, |Δc| = 0.9 px
Line 2	m = -1.13, c = 447.0 px	m = -1.13, c = 450.0 px	m = -1.13, c = 448.2 px	|Δm| = 0.00, |Δc| = 1.2 px
Line 3	m = 2.21, c = -190.0 px	m = 2.17, c = -180.0 px	m = 2.14, c = -179.8 px	|Δm| = 0.07, |Δc| = 10.2 px